



FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

2024 – 2025 SPRING

PROGRAMMING LANGUAGE PROJECT: YEM

STUDENT ID	NAME/SURNAME
220315016	Yusuf BARIŞ
220315040	Emine ÇETİN
220315082	Mustafa Furkan YILMAZ

Grammar of the Language – In BNF Format

<program> ::= { <statement> } <EOF>

<statement> ::= <yaz_statement> "_"
 | <assignment_statement> "_"
 | <for_loop_statement>
 | <while_loop_statement>

<yaz_statement> ::= "yaz" "(" <expression> ")"

<assignment_statement> ::= <identifier> "=" <expression>

<for_loop_statement> ::= "her" "("

<assignment_statement_no_semicolon> ";"
 <condition_expression> ";"
 <assignment_statement_no_semicolon> ")" "{"
 { <loop_body_statement> } "}"

<while_loop_statement> ::= "sürece" "(" <condition_expression> ")"
"{"

 { <loop_body_statement> } "}"

<loop_body_statement> ::= <yaz_statement_inside_loop>

 | <assignment_statement_inside_loop>

<yaz_statement_inside_loop> ::= "yaz" "(" <expression> ")"

<assignment_statement_inside_loop> ::= <identifier> "="
<expression>

<assignment_statement_no_semicolon> ::= <identifier> "="
<expression>

<condition_expression> ::= <expression> <comparison_operator>
<expression>

<expression> ::= <term> { ("+" | "-" | "*" | "/") <term> }

<term> ::= <identifier>

| <integer_literal>

| <string_literal>

| "(" <expression> ")"

<comparison_operator> ::= "==" | "!=" | "<" | ">" | "<=" | ">="

<identifier> ::= TOKEN_IDENTIFIER

<integer_literal> ::= TOKEN_INTEGER_LITERAL

<string_literal> ::= TOKEN_STRING_LITERAL

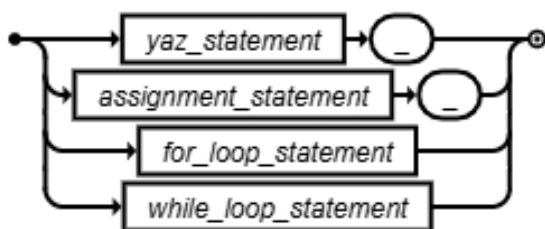
<EOF> ::= TOKEN_EOF

BASIC STATE DIAGRAMS

program:



statement:



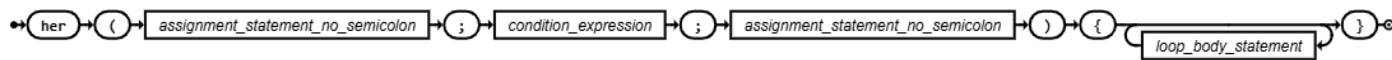
yaz_statement:



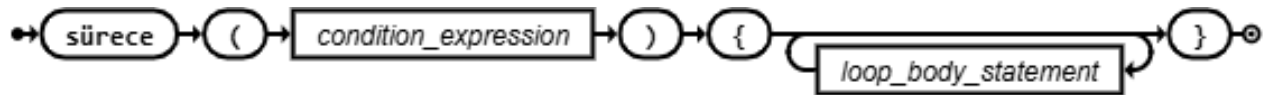
assignment_statement:



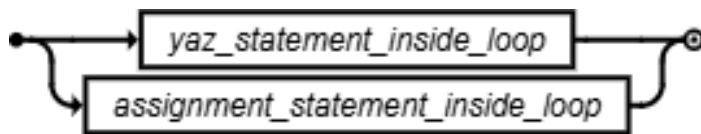
for_loop_statement:



while_loop_statement:



loop_body_statement:



yaz_statement_inside_loop:



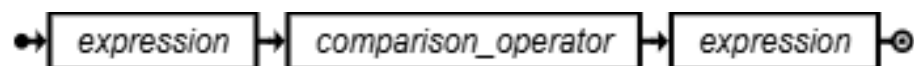
assignment_statement_inside_loop:



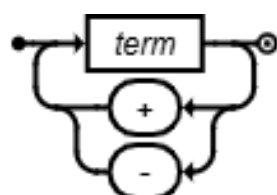
assignment_statement_no_semicolon:



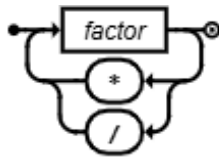
condition_expression:



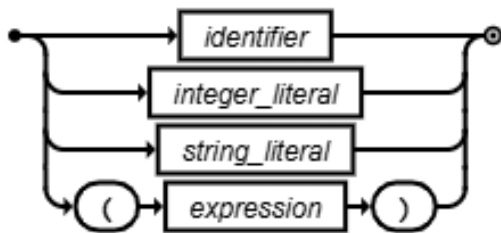
expression:



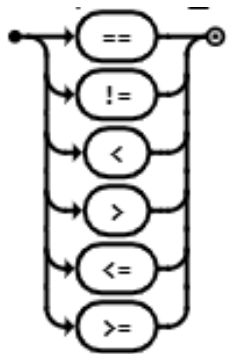
term:



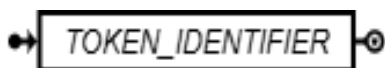
factor:



comparison_operator:



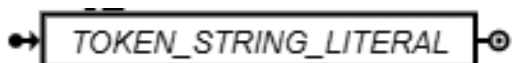
identifier:



integer_literal:



string_literal:



EOF:



Look-Up Table (Reserved Words and Tokens)

The YEM programming language supports Turkish-specific syntax and includes special characters. The following table outlines the reserved words and symbols recognized by the lexer.

Category	Token Type	Lexeme (Value)	Description
Keywords	TOKEN_KEYWORD	yaz	Used for printing output.
Keywords	TOKEN_KEYWORD	her	Used to initiate a for loop.
Keywords	TOKEN_KEYWORD	sürece	Used to initiate a while loop.
Identifiers	TOKEN_IDENTIFIER	(variable name)	User-defined, includes letters/digits/Turkish chars
Literals	TOKEN_INTEGER_LITERAL	123	Integer value
Operators	TOKEN_OPERATOR	=, +, -, *, /	Assignment, arithmetic, concat
Operators	TOKEN_OPERATOR	==, !=, <, >, <=, >=	Comparison ops
Separators	TOKEN_SEPARATOR	(,), {, }, ;	Grouping, blocks, loop structure
Special	TOKEN_UNDERLINE	_	Statement terminator
Special	TOKEN_EOF	null	End of input

Detailed Explanation of Java Classes

Lexer.java – Class Explanation

This class performs **lexical analysis** — it reads the source code and converts it into a list of tokens.

Fields:

- String source: The source code to be analyzed.
- int currentPosition, currentLine, currentColumn: Track the current scanning position.
- List<Token> tokens: Stores the generated tokens.
- Map<String, TokenType> KEYWORDS: Maps keywords like yaz, her, sürece to token types.

Functions:

Lexer(String source)

Constructor. Initializes the source code string to be lexed.

char advance()

Moves to the next character and updates line/column counters.
Returns the character read.

char peek()

Peeks at the current character without consuming it.

char peekNext(int offset)

Peeks ahead by a given offset. Useful for multi-character lookaheads.

void skipWhitespace()

Skips whitespace characters like spaces, tabs, and newlines.

Token createToken(TokenType type, String value)

Creates a token with given type and value and tracks its location (line/column).

void addToken(Token token)

Adds the given token to the tokens list.

void tokenize()

This is the **main lexical analyzer** function.

- Loops over each character of the source.
- Recognizes tokens such as keywords (yaz, her, etc.), operators (+, -, =, etc.), identifiers, and numbers.
- Skips over whitespace.
- Handles Turkish characters (ş, ü, ö, ç, ğ, ı).
- At the end, adds an EOF (End of File) token.

List<Token> getTokens()

Returns the list of generated tokens after calling tokenize().

Parser.java – Class Explanation

This class performs **syntax analysis** — it reads the tokens from the lexer and interprets them according to language grammar rules.

Fields:

- List<Token> tokens: The list of tokens to parse.
- int currentTokenIndex: The current position in the token list.
- Map<String, Object> environment: A simulated variable environment.

Functions:

Parser(List<Token> tokens)

Constructor. Initializes the parser with a list of tokens.

Token currentToken()

Returns the current token in the list.

Token peekNextToken()

Looks ahead to the next token without advancing.

Token consumeToken()

Advances to the next token, unless it's EOF. Returns the consumed token.

Token match(TokenType expectedType)

Consumes the token only if its type matches the expected type; throws a syntax error otherwise.

Token match(TokenType expectedType, String expectedValue)

Same as above, but checks both token type and specific value.

void parse()

The main entry point for parsing.

- Loops through all tokens and parses statements one by one.

void parseStatement()

Parses a single statement: yaz, her, sürece, assignments.

void parsePrintStatement()

Handles yaz statements (print).

void parseForLoop()

Handles her loops.

void parseWhileLoop()

Handles sürece (while) loops.

void parseAssignment()

Parses assignment expressions.

Object evaluateExpression()

Evaluates expressions with operators like +, -, *, /.

Object evaluatePrimary()

Evaluates literals or variables.

Token.java – Class Explanation

Represents a single token with type, value, and source location info.

Fields:

- TokenType type: The category of token (keyword, identifier, operator, etc.).
- String value: The string value of the token.
- int line: The line number in the source code.
- int column: The column number.

Constructor:

```
public Token(TokenType type, String value, int line, int column)
```

Initializes a new token with type, value, and position info.

toString() method

Provides a formatted string representation of the token, used for debugging or logging.

TokenType.java – Class Explanation

The TokenType enum defines the different **types of tokens** that can be recognized during lexical analysis. This classification helps the Lexer and Parser understand the role of each token in the source code.

Token Type	Description
TOKEN_KEYWORD	Reserved words that have a predefined meaning in the language. In your language YEM, examples include yaz, her, sürece.
TOKEN_IDENTIFIER	Variable names or function names defined by the programmer. For example: sayi, mesaj.
TOKEN_INTEGER_LITERAL	Numeric constants, e.g., 42, 100, 0.
TOKEN_STRING_LITERAL	Text enclosed in quotes, e.g., "Merhaba", "Toplam: ".
TOKEN_OPERATOR	Symbols that represent operations: +, -, *, /, = etc.
TOKEN_SEPARATOR	Punctuation used to separate statements or expressions, e.g., (,), {, }, ;.
TOKEN_EOF	Stands for End of File . It is a special token that marks the end of the input stream.
TOKEN_UNDERLINE	Custom type for handling the underscore _ character, possibly used in variable names or as a standalone token.

Why This Enum Matters

During the **lexical analysis**, each piece of the source code is matched to one of these types. This enum helps:

- Differentiate between data and instructions (e.g., keywords vs. literals)
- Validate syntax rules in the Parser
- Make code generation or interpretation easier by classifying components

YemInterpreter.java – Class Explanation

This is the **main class** that acts as the interpreter for the YEM programming language. It coordinates the reading of source code, tokenization (lexical analysis), parsing (syntax analysis), and execution.

Determine File Path

- If a command-line argument is provided, it uses that as the source file path.
- Otherwise, it defaults to a hardcoded .yem file path.

Read Source Code

- Reads the content of the .yem file as a UTF-8 string.
- If reading fails, an error message is printed and the program exits.

Lexical Analysis

- Passes the source code to the **Lexer**, which converts it into a list of tokens.
- Prints each token.
- If an error occurs (e.g., unrecognized character), it prints a **Lexical Analysis Error** and exits.

Syntax Analysis & Execution

- Sends the list of tokens to the **Parser**.
- The parser checks the syntax and interprets the code (e.g., executes loops, assignments, print statements).
- If an error occurs (e.g., unexpected token or syntax), it prints a **Parsing Error**.

Console Output Structure

Throughout the program, nicely formatted output is printed to help with debugging and understanding the program's behavior:

- --- Kaynak Kod --- : Prints raw source code
- --- Belirteçler (Tokenlar) --- : Prints all tokens
- --- Ayırıştırma Çıktısı --- : Shows parsing and execution output

Summary

Step	Action
1. File Reading	Reads .yem file using UTF-8
2. Lexical Analysis	Creates tokens from the source code
3. Syntax Analysis	Parses and executes the tokens
4. Output	Prints results or any errors detected

This class is essential as it **glues all components** (Lexer, Token, Parser) together and provides an interface to run YEM programs from .yem files.

Example Source Code Files

Below are **four example source files** written in the **YEM programming language**. Each file demonstrates different features such as variable assignments, arithmetic expressions, loops, and output commands.

Example 1 – Basic Output and Arithmetic

Filename: ornek1.yem

INPUTS:

```
--- Kaynak Kod ---
yaz ("Yem dili ile ilk kodum!")_

mesaj = "Merhaba Dünya"_
sayi = 123_

yaz (mesaj)_
yaz (sayi)_

toplamaSonucu = 50 + 25_
yaz ("Toplama Sonucu:")_
yaz (toplamaSonucu)_

carpmaSonucu = 10 * 4_
yaz ("Çarpma Sonucu:")_
yaz (carpmaSonucu)_

degiskenMesaj = "Degisken"_
degiskenSayi = 7_
yaz (degiskenMesaj + ": " + degiskenSayi)_
-----
```

OUTPUTS:

```
--- Belirteçler (Tokenlar) ---
Token{type=TOKEN_KEYWORD, value='yaz', line=1, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=1, column=5}
Token{type=TOKEN_STRING_LITERAL, value='Yem dili ile ilk kodum!', line=1, column=6}
Token{type=TOKEN_SEPARATOR, value=')', line=1, column=31}
Token{type=TOKEN_UNDERLINE, value='_', line=1, column=32}
Token{type=TOKEN_IDENTIFIER, value='mesaj', line=3, column=1}
Token{type=TOKEN_OPERATOR, value='=', line=3, column=7}
Token{type=TOKEN_STRING_LITERAL, value='Merhaba Dünya', line=3, column=9}
Token{type=TOKEN_UNDERLINE, value='_', line=3, column=24}
Token{type=TOKEN_IDENTIFIER, value='sayi', line=4, column=1}
Token{type=TOKEN_OPERATOR, value='=', line=4, column=6}
Token{type=TOKEN_INTEGER_LITERAL, value='123', line=4, column=8}
Token{type=TOKEN_UNDERLINE, value='_', line=4, column=11}
Token{type=TOKEN_KEYWORD, value='yaz', line=6, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=6, column=5}
Token{type=TOKEN_IDENTIFIER, value='mesaj', line=6, column=6}
Token{type=TOKEN_SEPARATOR, value=')', line=6, column=11}
Token{type=TOKEN_UNDERLINE, value='_', line=6, column=12}
Token{type=TOKEN_KEYWORD, value='yaz', line=7, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=7, column=5}
Token{type=TOKEN_IDENTIFIER, value='sayi', line=7, column=6}
Token{type=TOKEN_SEPARATOR, value=')', line=7, column=10}
Token{type=TOKEN_UNDERLINE, value='_', line=7, column=11}
Token{type=TOKEN_IDENTIFIER, value='toplamaSonucu', line=9, column=1}
Token{type=TOKEN_OPERATOR, value='=', line=9, column=15}
Token{type=TOKEN_INTEGER_LITERAL, value='50', line=9, column=17}
Token{type=TOKEN_OPERATOR, value='+', line=9, column=20}
Token{type=TOKEN_INTEGER_LITERAL, value='25', line=9, column=22}
Token{type=TOKEN_UNDERLINE, value='_', line=9, column=24}
Token{type=TOKEN_KEYWORD, value='yaz', line=10, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=10, column=5}
Token{type=TOKEN_STRING_LITERAL, value='Toplama Sonucu:', line=10, column=6}
Token{type=TOKEN_SEPARATOR, value=')', line=10, column=23}
Token{type=TOKEN_UNDERLINE, value='_', line=10, column=24}
Token{type=TOKEN_KEYWORD, value='yaz', line=11, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=11, column=5}
Token{type=TOKEN_IDENTIFIER, value='toplamaSonucu', line=11, column=6}
Token{type=TOKEN_SEPARATOR, value=')', line=11, column=19}
Token{type=TOKEN_UNDERLINE, value='_', line=11, column=20}
Token{type=TOKEN_IDENTIFIER, value='carpmaSonucu', line=13, column=1}
Token{type=TOKEN_OPERATOR, value='=', line=13, column=14}
Token{type=TOKEN_INTEGER_LITERAL, value='10', line=13, column=16}
Token{type=TOKEN_OPERATOR, value='*', line=13, column=19}
Token{type=TOKEN_INTEGER_LITERAL, value='4', line=13, column=21}
Token{type=TOKEN_UNDERLINE, value='_', line=13, column=22}
Token{type=TOKEN_KEYWORD, value='yaz', line=14, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=14, column=5}
Token{type=TOKEN_STRING_LITERAL, value='Çarpma Sonucu:', line=14, column=6}
Token{type=TOKEN_SEPARATOR, value=')', line=14, column=22}
Token{type=TOKEN_UNDERLINE, value='_', line=14, column=23}
Token{type=TOKEN_KEYWORD, value='yaz', line=15, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=15, column=5}
Token{type=TOKEN_IDENTIFIER, value='carpmaSonucu', line=15, column=6}
Token{type=TOKEN_SEPARATOR, value=')', line=15, column=18}
Token{type=TOKEN_UNDERLINE, value='_', line=15, column=19}
Token{type=TOKEN_IDENTIFIER, value='degiskenMesaj', line=17, column=1}
Token{type=TOKEN_OPERATOR, value='=', line=17, column=15}
Token{type=TOKEN_STRING_LITERAL, value='Degisken', line=17, column=17}
Token{type=TOKEN_UNDERLINE, value='_', line=17, column=27}
Token{type=TOKEN_IDENTIFIER, value='degiskenSayi', line=18, column=1}
Token{type=TOKEN_OPERATOR, value='=', line=18, column=14}
Token{type=TOKEN_INTEGER_LITERAL, value='7', line=18, column=16}
Token{type=TOKEN_UNDERLINE, value='_', line=18, column=17}
Token{type=TOKEN_KEYWORD, value='yaz', line=19, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=19, column=5}
Token{type=TOKEN_IDENTIFIER, value='degiskenMesaj', line=19, column=6}
Token{type=TOKEN_OPERATOR, value='+', line=19, column=20}
Token{type=TOKEN_STRING_LITERAL, value=': ', line=19, column=22}
Token{type=TOKEN_OPERATOR, value='+', line=19, column=27}
Token{type=TOKEN_IDENTIFIER, value='degiskenSayi', line=19, column=29}
Token{type=TOKEN_SEPARATOR, value=')', line=19, column=41}
Token{type=TOKEN_UNDERLINE, value='_', line=19, column=42}
Token{type=TOKEN_EOF, value='null', line=19, column=43}
```

```
--- Ayırıştırma Çıktısı ---
Parsing 'Yaz'statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Çıktı: Yem dili ile ilk kodum!

Parsing 'Atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Atama: mesaj = Merhaba Dünya

Parsing 'Atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Atama: sayi = 123

Parsing 'Yaz'statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Çıktı: Merhaba Dünya

Parsing 'Yaz'statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Çıktı: 123

Parsing 'Atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...

>>> Atama: toplamaSonucu = 75

Parsing 'Yaz'statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Çıktı: Toplama Sonucu:
```



```

Parsing 'Yaz'statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Çıktı: 75

Parsing 'Atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...

>>> Atama: carpmaSonucu = 40

Parsing 'Yaz'statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Çıktı: Çarpma Sonucu:

Parsing 'Yaz'statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Çıktı: 40

Parsing 'Atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Atama: degiskenMesaj = Degisken

Parsing 'Atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Atama: degiskenSayi = 7

Parsing 'Yaz'statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...
    Term değerlendiriliyor...

>>> Çıktı: Degisken: 7

-----

- Ayırıştırma ve çalıştırma başarıyla tamamlandı.
-----

```

Explanation:

- Defines and prints strings and integers.
- Performs **addition** and **multiplication**.
- Demonstrates **string concatenation** with variables.
- Uses the yaz() function for output (similar to print() in other languages).

Example 2 – Loop Structures

Filename: ornek2.yem

INPUTS:

```
--- Kaynak Kod ---
yaz ("Her Döngüsü Örneği")_
her (i = 0; i < 3; i = i + 1) {
    yaz ("Her döngüsü adım: " + i)
}

yaz ("Her Döngüsü Bitti")_

yaz ("Süreçe Döngüsü Örneği")_
j = 0_
sürece (j < 3) {
    yaz ("Süreçe döngüsü adım: " + j)
    j = j + 1
}

yaz ("Süreçe Döngüsü Bitti")_
-----
```

OUTPUTS:

```
--- Belirteçler (Tokenlar) ---
Token{type=TOKEN_KEYWORD, value='yaz', line=1, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=1, column=5}
Token{type=TOKEN_STRING_LITERAL, value='Her Döngüsü Örneği', line=1, column=6}
Token{type=TOKEN_SEPARATOR, value=')', line=1, column=26}
Token{type=TOKEN_UNDERLINE, value='_', line=1, column=27}
Token{type=TOKEN_KEYWORD, value='her', line=2, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=2, column=5}
Token{type=TOKEN_IDENTIFIER, value='i', line=2, column=6}
Token{type=TOKEN_OPERATOR, value='=', line=2, column=8}
Token{type=TOKEN_INTEGER_LITERAL, value='0', line=2, column=10}
Token{type=TOKEN_SEPARATOR, value=';', line=2, column=11}
Token{type=TOKEN_IDENTIFIER, value='i', line=2, column=13}
Token{type=TOKEN_OPERATOR, value='<', line=2, column=15}
Token{type=TOKEN_INTEGER_LITERAL, value='3', line=2, column=17}
Token{type=TOKEN_SEPARATOR, value=';', line=2, column=18}
Token{type=TOKEN_IDENTIFIER, value='i', line=2, column=20}
Token{type=TOKEN_OPERATOR, value='=', line=2, column=22}
Token{type=TOKEN_IDENTIFIER, value='i', line=2, column=24}
Token{type=TOKEN_OPERATOR, value='+', line=2, column=26}
Token{type=TOKEN_INTEGER_LITERAL, value='1', line=2, column=28}
Token{type=TOKEN_SEPARATOR, value=')', line=2, column=29}
Token{type=TOKEN_SEPARATOR, value='{', line=2, column=31}
Token{type=TOKEN_KEYWORD, value='yaz', line=3, column=4}
Token{type=TOKEN_SEPARATOR, value='(', line=3, column=8}
Token{type=TOKEN_STRING_LITERAL, value='Her döngüsü adım: ', line=3, column=9}
Token{type=TOKEN_OPERATOR, value='+', line=3, column=30}
Token{type=TOKEN_IDENTIFIER, value='i', line=3, column=32}
Token{type=TOKEN_SEPARATOR, value=')', line=3, column=33}
Token{type=TOKEN_SEPARATOR, value='}', line=4, column=1}
Token{type=TOKEN_KEYWORD, value='yaz', line=6, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=6, column=5}
Token{type=TOKEN_STRING_LITERAL, value='Her Döngüsü Bitti', line=6, column=6}
```

```
Token{type=TOKEN_SEPARATOR, value=')', line=6, column=25}
Token{type=TOKEN_UNDERLINE, value='_', line=6, column=26}
Token{type=TOKEN_KEYWORD, value='yaz', line=8, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=8, column=5}
Token{type=TOKEN_STRING_LITERAL, value='Sürece Döngüsü Örneği', line=8, column=6}
Token{type=TOKEN_SEPARATOR, value=')', line=8, column=29}
Token{type=TOKEN_UNDERLINE, value='_', line=8, column=30}
Token{type=TOKEN_IDENTIFIER, value='j', line=9, column=1}
Token{type=TOKEN_OPERATOR, value='=', line=9, column=3}
Token{type=TOKEN_INTEGER_LITERAL, value='0', line=9, column=5}
Token{type=TOKEN_UNDERLINE, value='_', line=9, column=6}
Token{type=TOKEN_KEYWORD, value='sürece', line=10, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=10, column=8}
Token{type=TOKEN_IDENTIFIER, value='j', line=10, column=9}
Token{type=TOKEN_OPERATOR, value='<', line=10, column=11}
Token{type=TOKEN_INTEGER_LITERAL, value='3', line=10, column=13}
Token{type=TOKEN_SEPARATOR, value=')', line=10, column=14}
Token{type=TOKEN_SEPARATOR, value='{', line=10, column=16}
Token{type=TOKEN_KEYWORD, value='yaz', line=11, column=4}
Token{type=TOKEN_SEPARATOR, value='(', line=11, column=8}
Token{type=TOKEN_STRING_LITERAL, value='Sürece döngüsü adım: ', line=11, column=9}
Token{type=TOKEN_OPERATOR, value='+', line=11, column=33}
Token{type=TOKEN_IDENTIFIER, value='j', line=11, column=35}
Token{type=TOKEN_SEPARATOR, value=')', line=11, column=36}
Token{type=TOKEN_IDENTIFIER, value='j', line=12, column=4}
Token{type=TOKEN_OPERATOR, value='=', line=12, column=6}
Token{type=TOKEN_IDENTIFIER, value='j', line=12, column=8}
Token{type=TOKEN_OPERATOR, value='+', line=12, column=10}
Token{type=TOKEN_INTEGER_LITERAL, value='1', line=12, column=12}
Token{type=TOKEN_SEPARATOR, value=')', line=13, column=1}
Token{type=TOKEN_KEYWORD, value='yaz', line=15, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=15, column=5}
Token{type=TOKEN_STRING_LITERAL, value='Sürece Döngüsü Bitti', line=15, column=6}
Token{type=TOKEN_SEPARATOR, value=')', line=15, column=28}
Token{type=TOKEN_UNDERLINE, value='_', line=15, column=29}
Token{type=TOKEN_EOF, value='null', line=15, column=30}
-----
```

```
--- Ayırıştırma Çıktısı ---
Parsing 'Yaz' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Çıktı: Her Döngüsü Örneği

Parsing 'Her' statement
  For döngüsü başlığı - başlangıç ataması:
    Expression değerlendiriliyor..
      Term değerlendiriliyor...
    >>> Döngü Başlığı Ataması: i = 0
    Condition yapısı ayrıştırılıyor (sadece ilerleme için)...
    Expression değerlendiriliyor..
      Term değerlendiriliyor...
    Expression değerlendiriliyor..
      Term değerlendiriliyor...
    Expression değerlendiriliyor..
      Term değerlendiriliyor...
      Term değerlendiriliyor...
    >>> Döngü Başlığı Ataması: i = 1
    Condition değerlendiriliyor...
    Expression değerlendiriliyor..
      Term değerlendiriliyor...
    Expression değerlendiriliyor..
      Term değerlendiriliyor...
Parsing döngü içindeki 'yaz' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...

>>> Çıktı: Her döngüsü adım: 1

  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...
  >>> Döngü Başlığı Ataması: i = 2
  Condition değerlendiriliyor...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
Parsing döngü içindeki 'yaz' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...

>>> Çıktı: Her döngüsü adım: 2
```

```
Expression değerlendiriliyor..
  Term değerlendiriliyor...
  Term değerlendiriliyor...
>>> Döngü Başlığı Ataması: i = 3
Condition değerlendiriliyor...
Expression değerlendiriliyor..
  Term değerlendiriliyor...
Expression değerlendiriliyor..
  Term değerlendiriliyor...
Parsing 'Yaz'statement
  Expression değerlendiriliyor..
  Term değerlendiriliyor...

>>> Çıktı: Her Döngüsü Bitti

Parsing 'Yaz'statement
  Expression değerlendiriliyor..
  Term değerlendiriliyor...

>>> Çıktı: Sürece Döngüsü Örneği

Parsing 'Atama' statement
  Expression değerlendiriliyor..
  Term değerlendiriliyor...

>>> Atama: j = 0

Parsing 'Sürece' statement
  Condition yapısı ayrıştırılıyor (sadece ilerleme için)...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
  Condition değerlendiriliyor...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
Parsing döngü içindeki 'yaz' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...

>>> Çıktı: Sürece döngüsü adım: 0
```

```
Parsing döngü içindeki 'atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...
```

```
>>> Atama: j = 1
```

```
  Condition değerlendiriliyor...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
Parsing döngü içindeki 'yaz' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...
```

```
>>> Çıktı: Sürece döngüsü adım: 1
```

```
Parsing döngü içindeki 'atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...
```

```
>>> Atama: j = 2
```

```
  Condition değerlendiriliyor...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
Parsing döngü içindeki 'yaz' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...
```

```
>>> Çıktı: Sürece döngüsü adım: 2
```

```
Parsing döngü içindeki 'atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...
```

```
>>> Atama: j = 3
```

```
  Condition değerlendiriliyor...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
Parsing 'Yaz' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
```

```
>>> Çıktı: Sürece Döngüsü Bitti
```

```
-----
```

```
Ayırıştırma ve çalıştırma başarıyla tamamlandı.
```

Explanation:

- Demonstrates two types of loops:
 - her loop → similar to a for loop.
 - sürece loop → similar to a while loop.
- Shows loop variable initialization, conditions, and updates.
- Uses the yaz() function to print loop steps.

Example 3 – Runtime Error

Filename: ornek3.yem

INPUTS:

```
--- Kaynak Kod ---
x = 10_
y = 0_

sonuc = x / y_
yaz (sonuc)_
```

OUTPUTS:

```
--- Belirteçler (Tokenlar) ---
Token(type=TOKEN_IDENTIFIER, value='x', line=1, column=1)
Token(type=TOKEN_OPERATOR, value='=', line=1, column=3)
Token(type=TOKEN_INTEGER_LITERAL, value='10', line=1, column=5)
Token(type=TOKEN_UNDERLINE, value='_', line=1, column=7)
Token(type=TOKEN_IDENTIFIER, value='y', line=2, column=1)
Token(type=TOKEN_OPERATOR, value='=', line=2, column=3)
Token(type=TOKEN_INTEGER_LITERAL, value='0', line=2, column=5)
Token(type=TOKEN_UNDERLINE, value='_', line=2, column=6)
Token(type=TOKEN_IDENTIFIER, value='sonuc', line=4, column=1)
Token(type=TOKEN_OPERATOR, value='=', line=4, column=7)
Token(type=TOKEN_IDENTIFIER, value='x', line=4, column=9)
Token(type=TOKEN_OPERATOR, value='/', line=4, column=11)
Token(type=TOKEN_IDENTIFIER, value='y', line=4, column=13)
Token(type=TOKEN_UNDERLINE, value='_', line=4, column=14)
Token(type=TOKEN_KEYWORD, value='yaz', line=5, column=1)
Token(type=TOKEN_SEPARATOR, value='(', line=5, column=5)
Token(type=TOKEN_IDENTIFIER, value='sonuc', line=5, column=6)
Token(type=TOKEN_SEPARATOR, value=')', line=5, column=11)
Token(type=TOKEN_UNDERLINE, value='_', line=5, column=12)
Token(type=TOKEN_EOF, value='null', line=5, column=13)
-----

--- Ayırıştırma Çıktısı ---
Parsing 'Atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Atama: x = 10

Parsing 'Atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Atama: y = 0

Parsing 'Atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...
    Term değerlendiriliyor...
Runtime Hatası: Sıfıra bölme hatası. Konum: 4. satır, 14. sütun.
-----
```

Explanation:

- Demonstrates **division operation**.
- Deliberately causes a **division by zero**, which can be used to test error handling.
- The interpreter is expected to handle or display an appropriate error message.

Example 4 – Syntax Error

Filename: ornek4.yem

INPUTS/OUTPUTS:

```
--- Kaynak Kod ---
yaz ("Bu doğru yazılmış bir deyim")_

hataOlacakDeyisken = 0_
yaz (hataOlacakDeyisken_
-----

--- Belirteçler (Tokenlar) ---
Token{type=TOKEN_KEYWORD, value='yaz', line=1, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=1, column=5}
Token{type=TOKEN_STRING_LITERAL, value='Bu doğru yazılmış bir deyim', line=1, column=6}
Token{type=TOKEN_SEPARATOR, value=')', line=1, column=35}
Token{type=TOKEN_UNDERLINE, value='_', line=1, column=36}
Token{type=TOKEN_IDENTIFIER, value='hataOlacakDeyisken', line=3, column=1}
Token{type=TOKEN_OPERATOR, value='=', line=3, column=20}
Token{type=TOKEN_INTEGER_LITERAL, value='0', line=3, column=22}
Token{type=TOKEN_UNDERLINE, value='_', line=3, column=23}
Token{type=TOKEN_KEYWORD, value='yaz', line=4, column=1}
Token{type=TOKEN_SEPARATOR, value='(', line=4, column=5}
Token{type=TOKEN_IDENTIFIER, value='hataOlacakDeyisken', line=4, column=6}
Token{type=TOKEN_UNDERLINE, value='_', line=4, column=24}
Token{type=TOKEN_EOF, value='null', line=4, column=25}
-----

--- Ayırıştırma Çıktısı ---
Parsing 'Yaz' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Çıktı: Bu doğru yazılmış bir deyim

Parsing 'Atama' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Atama: hataOlacakDeyisken = 0

Parsing 'Yaz' statement
  Expression değerlendiriliyor..
    Term değerlendiriliyor...

>>> Çıktı: 0

Syntax Hatası: Beklenen token -> ' ) ' ama bulunan token -> ' _ ' 4. satır, 24. sütun.
-----
```

Explanation:

For the 'yaz' function to work, the parenthesis must be closed, but here, an underscore ('_') is used without closing the parenthesis